

Doc-Doctor 用户使用手册

1. 简介

1.1 什么是 Doc-Doctor

Doc-Doctor 是一款 Visual Studio Code 扩展插件，专门用于自动检测 C/C++ 项目中函数注释的完整性问题。它能够：

- 自动扫描项目中的 C/C++ 源文件
- 检测函数注释是否符合 Doxygen 规范
- 识别缺失的参数说明、返回值说明、函数功能描述
- 基于 Git 历史检测函数内容变更，提醒更新注释
- 提供 AI 辅助生成和更新函数注释的能力

1.2 适用场景

- **代码审查**：快速发现注释不完整的函数
 - **代码维护**：检测函数实现变更但注释未同步的情况
 - **团队协作**：统一代码注释规范，提升代码可读性
 - **项目重构**：批量检查历史代码的注释质量
-

2. 安装与配置

2.1 安装扩展

方法一：从 VS Code 市场安装（尚未上架）

1. 打开 VS Code
2. 点击左侧活动栏的扩展图标（或按 `Ctrl+Shift+X`）
3. 搜索 "Doc-Doctor"
4. 点击"安装"按钮

方法二：从源码安装

1. 克隆仓库：

```
git clone <repository-url>
cd Doc-Doctor/doc-doctor
```

2. 安装依赖：

```
npm install
```

3. 编译 TypeScript：

```
npm run compile
```

4. 按 **F5** 启动调试模式

2.2 基本配置

扩展安装后即可使用，无需额外配置。如需自定义检查规则，可进入设置页面配置。

3. 快速开始

3.1 打开扩展

1. 在 VS Code 活动栏中点击 **Doc-Doctor** 图标
2. 侧边栏会显示 Doc-Doctor 面板

3.2 执行检查

1. 点击 **"检查整个项目"** 按钮
2. 等待检查完成（进度会显示在输出日志中）
3. 查看问题列表

3.3 查看问题

- 问题列表会显示所有发现的注释问题
- 点击问题卡片可跳转到对应代码位置
- 使用搜索框和类型筛选快速定位问题

4. 功能详解

4.1 项目检查

4.1.1 检查整个项目

功能说明：扫描工作区内所有 `.c` 和 `.cpp` 文件，检测函数注释问题。

使用方法：

1. 在检查页点击 **"检查整个项目"** 按钮
2. 系统会自动扫描工作区文件
3. 检查过程中可以点击 **"取消检查"** 终止检查
4. 检查完成后，问题列表会自动更新

检查范围：

- 仅检查扩展名为 `.c` 和 `.cpp` 的文件
- 超过 1MB 的文件会自动跳过
- 无读取权限的文件会跳过并提示

4.1.2 检查单个文件

功能说明：选择单个文件进行检查。

使用方法：

- 1. 点击 "检查单个文件" 按钮
- 2. 在文件选择对话框中选择要检查的文件
- 3. 系统会解析该文件并显示问题

4.2 问题类型

系统会检测以下类型的问题：

问题类型	说明	示例
参数缺失	函数参数缺少 @param 说明	<code>int add(int a, int b)</code> 缺少 @param a 和 @param b
返回值缺失	非 void 函数缺少 @return 说明	<code>int calculate()</code> 缺少 @return
说明缺失	函数缺少 @brief 功能描述	函数注释中没有 @brief 或功能描述
变更警告	函数内容变更但注释未更新	函数实现已修改，但注释未同步
语法错误	文件存在编译错误	文件无法正确解析，存在语法错误

4.3 问题列表操作

4.3.1 搜索问题

功能说明：按函数名或文件路径搜索问题。

使用方法：

- 1. 在搜索框中输入关键词
- 2. 问题列表会实时过滤显示匹配的问题

搜索规则：

- 支持函数名搜索
- 支持文件路径搜索
- 搜索不区分大小写

4.3.2 筛选问题

功能说明：按问题类型筛选问题列表。

使用方法：

- 1. 点击类型下拉框
- 2. 选择要查看的问题类型
- 3. 问题列表会只显示该类型的问题

筛选选项：

- 所有类型（默认）
- 参数缺失
- 返回值缺失
- 说明缺失
- 变更警告
- 语法错误

4.3.3 标记问题状态

功能说明：将问题标记为"已完成"状态。

使用方法：

1. 点击问题卡片右上角的标记按钮（○）
2. 按钮会变为 ✓，表示已标记为完成
3. 已完成的问题会置底显示，透明度降低

说明：

- 问题状态会持久化保存
- 下次打开插件时会自动恢复状态
- 可以再次点击取消标记

4.3.4 跳转到代码

功能说明：点击问题卡片，自动跳转到对应代码位置。

使用方法：

1. 点击任意问题卡片
2. VS Code 会自动打开对应文件
3. 光标会定位到函数定义位置

说明：

- 如果文件已被删除，会显示错误提示
- 如果函数已被删除，会显示错误提示

4.4 AI 辅助修复

4.4.1 启用 AI 功能

前置条件：

1. 在设置页启用 "启用 AI 修复注释"
2. 配置 AI 服务地址和 API Key
3. 保存设置

4.4.2 使用 AI 修复

使用方法：

1. 在问题卡片上点击 **"AI修复注释"** 按钮（仅对问题类型 1/2/3/4 显示）
2. 系统会调用 AI 服务生成注释
3. 在预览界面查看新旧注释对比
4. 确认无误后点击 **"应用修改"**
5. 系统会将新注释写回源码文件
6. 问题状态会自动更新为"已完成"

注意事项：

- AI 功能需要网络连接
- 每次请求会发送当前函数的源码到 AI 服务
- 建议在应用修改前仔细检查生成的注释
- API Key 使用 SecretStorage 加密存储

4.5 设置配置

4.5.1 检查规则配置

检查 main 函数：

- 默认不检查 `main` 函数
- 如需检查，勾选此选项

启用 Git 变更警告：

- 默认启用
- 基于 Git 历史检测函数内容变更
- 需要工作区为 Git 仓库且至少两条提交记录

4.5.2 AI 辅助功能配置

启用 AI 修复注释：

- 默认关闭
- 需要手动启用

AI 服务地址：

- 输入 AI 服务的 HTTP 接口地址
- 支持 OpenAI 兼容的 API 格式

API Key：

- 输入 AI 服务的访问密钥
- 使用 SecretStorage 加密存储

4.5.3 白名单配置

文件白名单：

- 每行一个路径，支持目录前缀匹配
- 匹配的文件会跳过检查
- 示例：`test/`、`vendor/`、`src/legacy/`

函数白名单：

- 支持两种格式：
 - `文件路径:函数名`（文件级白名单）
 - `函数名`（全局白名单）
- 示例：`src/legacy.c:oldFunction`、`globalHelper`

返回值类型白名单：

- 每行一个类型
- 匹配类型的函数会跳过检查
- 示例：`void`

4.5.4 保存设置

1. 修改配置后，点击 "保存设置" 按钮
2. 配置会保存到工作区的 `.vscode/settings.json`
3. 保存成功后会自动触发重新检查

5. 注释规范

5.1 Doxygen 注释格式

Doc-Doctor 检测 **Doxygen 风格** 的注释，标准格式如下：

```
/**
 * @brief 函数功能的简要说明
 * @param param1 参数1的说明
 * @param param2 参数2的说明
 * @return 返回值的说明
 */
int exampleFunction(int param1, char* param2) {
    // 函数实现
    return 0;
}
```

5.2 注释要求

必需元素：

- `@brief`：函数功能的简要说明（或首行描述）
- `@param`：每个参数都需要对应的 `@param` 说明

- `@return`：非 void 函数必须有 `@return` 说明

注释位置：

- 注释应紧邻函数定义之前
- 注释块与函数之间只能有空白字符

注释格式：

- 使用 `/** ... */` 或 `/* ... */` 格式
- 每行以 `*` 开头（可选）

5.3 示例

正确示例：

```
/**
 * @brief 计算两个整数的和
 * @param a 第一个加数
 * @param b 第二个加数
 * @return 两数之和
 */
int add(int a, int b) {
    return a + b;
}
```

错误示例：

```
// 缺少 @brief
/**
 * @param a 第一个加数
 * @param b 第二个加数
 * 缺少 @return
 */
int add(int a, int b) {
    return a + b;
}
```

6. 常见问题

6.1 检查相关问题

Q: 为什么某些文件没有被检查？

A: 可能的原因：

- 文件扩展名不是 `.c` 或 `.cpp`
- 文件大小超过 1MB
- 文件在文件白名单中
- 文件无读取权限

Q: 检查时间很长怎么办？

A:

- 大项目检查需要一定时间，请耐心等待
- 可以点击"取消检查"终止检查
- 检查过程中会显示进度信息

Q: 为什么某些函数没有被检查?

A: 可能的原因:

- 函数在函数白名单中
- 函数的返回类型在返回值类型白名单中
- 函数是 `main` 函数且未启用检查 `main` 函数选项

6.2 Git 相关问题

Q: Git 变更警告功能不可用?

A: 需要满足以下条件:

- 工作区是 Git 仓库 (存在 `.git` 目录)
- 当前分支至少有两条提交记录
- 在设置中启用了 Git 变更警告

Q: 为什么没有检测到函数变更?

A:

- 仅检测实质性变更 (忽略空白和注释变更)
- 新增函数不会生成变更警告
- 删除函数不会生成任何问题

6.3 AI 功能问题

Q: AI 修复注释按钮不可用?

A: 需要:

- 在设置中启用 AI 修复注释功能
- 配置正确的 AI 服务地址和 API Key
- 问题类型必须是 1/2/3/4 (语法错误不支持)

Q: AI 请求失败怎么办?

A: 可能的原因:

- 网络连接问题
- API Key 无效
- AI 服务地址错误
- 服务超时 (默认 60 秒)

解决方案:

- 检查网络连接

- 验证 API Key 是否正确
- 检查 AI 服务地址格式
- 查看输出日志中的错误信息

6.4 数据持久化问题

Q: 检查结果会保存吗？

A:

- 检查结果会保存到 `.doc-doctor/problems.db` 数据库文件
- 下次打开插件时会自动加载
- 问题状态（已完成）也会持久化保存

Q: 如何清除检查结果？

A:

- 执行新的检查会自动覆盖旧结果
- 可以手动删除 `.doc-doctor/problems.db` 文件

6.5 性能问题

Q: 检查速度慢怎么办？

A:

- 大项目检查需要一定时间，这是正常现象
- 可以配置文件白名单跳过不需要检查的文件
- 考虑使用增量检查（如果支持）

Q: 界面卡顿怎么办？

A:

- 问题数量过多时，前端最多显示 200 条
- 使用搜索和筛选功能减少显示数量
- 关闭不需要的标签页

7. 故障排除

7.1 扩展无法启动

症状： 点击 Doc-Doctor 图标后，侧边栏没有显示。

解决方案：

1. 检查扩展是否已正确安装
2. 查看 VS Code 输出面板中的错误信息
3. 尝试重新加载窗口（`Ctrl+R` 或 `Cmd+R`）
4. 重启 VS Code

7.2 检查功能不工作

症状： 点击检查按钮后没有反应。

解决方案：

1. 检查工作区是否已打开
2. 查看输出日志中的错误信息
3. 确认工作区中有 `.c` 或 `.cpp` 文件
4. 检查文件权限

7.3 数据库错误

症状： 提示数据库操作失败。

解决方案：

1. 检查 `.doc-doctor/problems.db` 文件权限
2. 尝试删除数据库文件，让系统重新创建
3. 检查磁盘空间是否充足

7.4 C++ DLL 加载失败

症状： 提示无法加载数据库 DLL。

解决方案：

1. 确认 `doc_doctor_db.dll` 文件存在于 `native/build/Release/` 目录
2. 检查 DLL 依赖项 (SQLite3) 是否可用
3. 重新编译 C++ 模块

8. 最佳实践

8.1 使用建议

1. **定期检查：** 建议在代码提交前执行检查
2. **配置白名单：** 将第三方库和测试文件加入白名单
3. **使用 AI 辅助：** 对于大量历史代码，可以使用 AI 功能批量修复
4. **关注变更警告：** 及时更新变更函数的注释

8.2 团队协作

1. **统一注释规范：** 团队统一使用 Doxygen 注释风格
2. **配置共享：** 将白名单配置提交到版本控制
3. **代码审查：** 在 Code Review 时关注注释完整性

8.3 性能优化

1. **合理使用白名单**：跳过不需要检查的文件和函数
 2. **增量检查**：只检查变更的文件（如果支持）
 3. **定期清理**：删除不需要的检查结果
-

9. 技术支持

9.1 获取帮助

- **GitHub Issues**：报告问题和建议
- **文档**：查看项目文档和 README
- **社区**：参与社区讨论

9.2 反馈问题

反馈问题时，请提供以下信息：

- VS Code 版本
 - 扩展版本
 - 操作系统版本
 - 问题描述和复现步骤
 - 错误日志（如有）
-

10. 更新日志

版本 1.0.0

- 初始版本发布
- 支持项目全量检查和单文件检查
- 支持问题列表展示和筛选
- 支持代码跳转和状态管理
- 支持 Git 变更警告
- 支持 AI 辅助修复注释
- 支持白名单配置